Object oriented Programming

Day - 18

```
class Cricketer {
    Constructor (name, role) {
        this.name = name;
        this.role = role;
        this.matchesplayed = 0;
        this.Stamina = 100
    }

    introduce () {
        return `${this.name} the ${this.role} | matchesplayed:
            ${this.matchesplayed} | Stamina : ${this.Stamina}`
    }

}

Const Player1 = new Cricketer ("virat", "batsman")

Const Player2 = new Cricketer ("Bumrah", "Bowler")

Console.log (Player1.hasOwnProperty("Name"));
```

↳ to check has Player1 has name Property so the answer is yes

- but we also could check with if Player1.name is accessible then property exist but in this way may be name Property inherit to some other object so hasOwnProperty is the best way

console.log (Player1.hasOwnProperty ("name")); // true

console.log (typeof Cricketer); // function

but we all may assume object but it function but this is same as yesterday class

```
class Debutant {
    constructor (name) {
        this.name = name
        this.walkout = () => `${this.name} walkout to bat for the 1st time`
    }
}

const debutant1 = new Debutant ("shubham")
const SomethingFromLastClass = debutant1.walkout

console.log (SomethingFromLastClass())
```

Shubham walkout to bat for the 1st time

```
const Debutant2 = new Debutant ("yashaoki");
console.log (debutant1.walkout == debutant2.walkout);
```

false

because both has memory stored but reference has been different

10 - Symbols

```
Const aadhar_of_mayur = Symbol("aadhar")
Const aadhar_of_piyush = Symbol("aadhar")
```

label
name से
एकसी पर
value unique होगी
guaranteed unique

```
Console.log(typeof aadhar_of_mayur);
Console.log(aadhar_of_mayur ≡ aadhar_of_piyush);
Console.log(aadhar_of_mayur.toString());
Console.log(aadhar_of_mayur.description);
```

⇓

Symbol

=> for

Symbol(aadhar)

aadhar.

```
Const non-indian = Symbol();
Console.log(nonindian.description).
```

↳ undefined

~~Const CitizenRecord = {~~

~~name: "Ved Pandey",~~

~~age: 21,~~

~~}~~

```
Const biometricHash = Symbol("biometricHash");
Const BloodGroup : Symbol("bloodgroup");

Const CitizenRecord = {
    name : "Ved Pandey",
    age : 21,
    [biometricHash] : "a7 y kh f kyr78 8dL",
    [Bloodgroup] : "O+"
}

Console.log(Object.keys(CitizenRecord));
```

↳ ['name', 'age']

↳ Symbol वाले 'an a key

Object के उन keys को visibility नहीं देगा

But आपको फिर करना होगा तब method होगा है,

Object.getOwnPropertySymbol(CitizenRecord)

↳ जब इसे log करोगे तो

[Symbol(biometricHash), Symbol(bloodgroup)]

Note : Object direct iterable नहीं होते

```
Const atiQueryBook = {
    queries : ["Infra budget", "Ration Card",
               "Education budget", "Startup laws"],
};

for (Const query of atiQueryBook) {
    Console.log(`Filling RTI: ${query}`);
}
```

↓

atiQueryBook is not iterable

Note But you can overrite means you can make it direct iterable with Symbol

```
Const atiQueryBook = {
    queries : ["Infra budget", "Ration Card",
               "Education budget", "Startup laws"],
    [Symbol.iterator]() {
        let index = 0;
        Const queries = this.queries;
        return {
            next() {
                if (index < queries.length) {
                    return {value: queries[index++], done: false};
                }
                return {value: undefined, done: true};
            }
        }
    }
};
```

for (Const query of myQueryBook) {

console.log(`Filling RTI : ${query}`);

⇓

Filling RTI : Infra budget

Filling RTI : Ration Card

Filling RTI : Education budget

Filling RTI : Startup Grants

Note देखा पता है हमने (for of) loop को direct object पर लगाया न कि queries पर तब भी पर Queries की element show हो रही है कि,

because मैंने logic में लिखा था

If anybody by mistake or any other wish to work to iterate object then they get iterate of queries

Note iteration में Return होने वाला next element होता है इसलिए next() का प्रयोग करते है.

```
Const governmentScheme = {
    name : "PM Kisan Yojna",
    People : 54,
    [Symbol.toPrimitive](hint) {
        if (hint === "string") return this.name;
        if (hint === "number") return 88;
    }
};
```

→ + चिह्न number में convert हो जाएगा

```
Consolelog (+ governmentScheme)
Consolelog (`${governmentScheme}`)
```

↓

88

PM Kisan Yojna

means जब कोई भी to convert governmentScheme object to number then get 88

and in string the get scheme name "PM Kisan Yojna

* Class function है सिर्फ Syntactical Sugar होती है।

* If any Code can't be able to run due to error then how can we prevent that Code so we need to learn Error Handling

17-try-catch.js

~~function navigation~~

```
function bootNavigation (mapLoaded) {
    try {
        Console.log ('Is navigation loaded : ${mapLoaded}');
        if (!mapLoaded) {
            throw new Error ("Map was not
                Passed in this function");
        }
        return "NAV_OK";
    } Catch (error){
        Console.log (error);
        Console.log ('Navigation Failed: ${error.message}');
    } finally{
        Console.log ("Navigation Sequence Completed");
    }
}
```

```
Const status1 = Boot Navigation (true);
Console.log('Result : ${status}');
```

⇓

```
To Navigation Loaded : true
Navigation Sequence Completed
Result : Nav_ok
```

If - promises JS

```
function PrepareOrderCB (dish, cb) {
  setTimeout (() => cb (null, {dish, status: "
                Prepared"}), 100);
}

function PickupOrderCB (order, cb) {
  setTimeout (() => cb (null, {...order, status:
    "picked_up"}), 100);
}
function deliverOrderCB (order, cb) {
  setTimeout (() => cb (null, {...order, status: "delivered"}),
              100);
}
```

Callback में error handle नहीं करना है तो 1st parameter null दे दो।

```
PrepareOrderCB ("Biryani", (err, order) => {
    if (err) return Console.log (err);
    PickupOrderCB (order, (err, order) => {
      if (err) return Console.log (err);
      deliverOrderCB (order, (err, order) => {
        if (err) return Console.log (err);
        Console.log ('${order.dish} : ${order.status}');
      })
    })
})
```

Promise के 3 step होते है।

① Pending ② Fulfilled ③ rejected

```
function pickupOrder(order){
  return new Promise((resolve, reject)=>{
    setTimeout(()=>{
      Console.log(`${order} is ready`)
      resolve({...order, status: "pickedup"
      },100)
    })
  }

function deliverorder(order){
  return new Promise((resolve reject)=>{
    setTimeout(()=>{
      Console.log(`${order} is ready`)
      resolve({...order, status: "delivered"})
    },100)
  })
}

function prepareorder(order){
  return new Promise((resolve, reject)=>{
    setTimeout(()=>{
      Console.log(`${order} is prepared`)
      resolved({...
```

```
function prepareOrder (dish) {
  return new Promise ((resolve, reject) => {
    setTimeout (() => {
      if!(dish) {
        reject (new Error ("No dish is there"))
        return
      }
      console.log (`${dish} is ready`)
      resolve ({dish, status: "prepared"})
      }, 100)
    })
  }


prepareOrder ("Paneer Tikka")
    .then (order => dispatchOrder (order))
    .then (order => deliverOrder (order))
    .catch (
```

koi bhi the error dega only at catch error catch मे, इसे error propagation होता है.

Note जो System के Error होते है वो try & Catch मे जाता है और जो बने बनाये Error हम देते है तो throw & Catch करता है